Task 1: Supervised ML Classifiers for Network Intrusion Detection

Comparative Implementation & Evaluation of Decision Tree, Random Forest, and Support Vector Machine on NSL-KDD

Student Name: Naman Katewa	**Roll Number:** 2401830002
Course: B.Sc.(H) Cybersecurity	**Subject:** Artificial Intelligence in Cyber Security
Dataset: NSL-KDD (25,192 records, 41 features)	**Validation Strategy:** 80% Train / 20% Test Stratified Split

1. Dataset Exploration & Feature Analysis

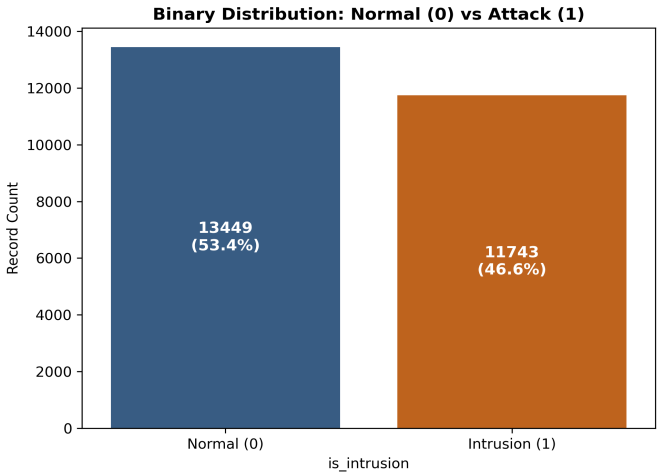
Network Intrusion Detection Systems (NIDS) inspect traffic flow attributes to identify unauthorized access, denial-of-service, and reconnaissance probes. The **NSL-KDD** dataset was developed to resolve major flaws in the legacy KDD'99 benchmark, eliminating duplicate connection records that previously biased classifier evaluations toward frequent flooding patterns.

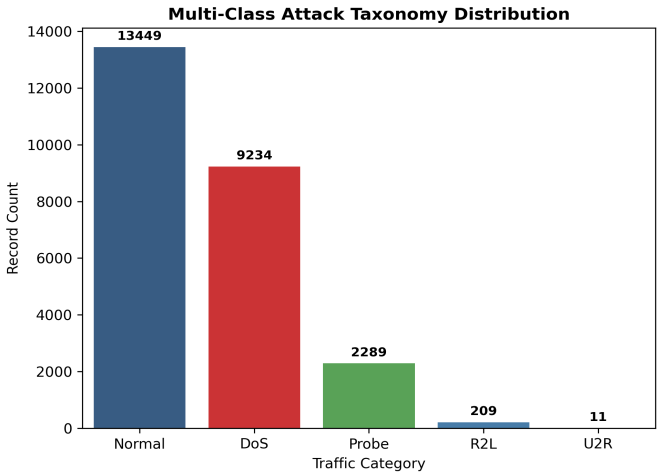
The analyzed benchmark contains **25,192 network connection records** across **41 input traffic features**. The feature space spans four primary categories:

- **Basic Features (1-9):** Intrinsic connection attributes including duration, protocol_type (TCP, UDP, ICMP), service (HTTP, FTP, SMTP, DNS), and flag (connection status such as SF, REJ, S0).
- **Content Features (10-22):** Payload-level inspection attributes including logged_in status, failed login attempts, root shell access, su command attempts, and compromised system files.
- **Time-based Traffic Features (23-31):** Windowed statistical properties over a 2-second interval, including connection count to the same host (count), same service (srv_count), and SYN error rates (serror_rate).
- **Host-based Traffic Features (32-41):** Destination host traffic history over a 100-connection window, capturing port scan and service probing patterns (dst_host_count, dst_host_srv_count, dst_host_same_src_port_rate).

Table 1: Class Distribution & Attack Taxonomy Breakdown

Category	Description	Attack Types Included	Count	Percentage
Normal	Legitimate network traffic	Benign connection sessions	13,449	53.39%
DoS	Denial of Service floods	neptune, smurf, back, teardrop, pod	9,234	36.65%
Probe	Surveillance and port scans	ipsweep, satan, portsweep, nmap	2,289	9.09%
R2L	Remote to Local unauthorized access	warezclient, guess_passwd, imap, phf	209	0.83%
U2R	User to Root privilege escalation	buffer_overflow, rootkit, loadmodule	11	0.04%
Total	**Complete Sampled Set**	**All traffic classes**	**25,192**	**100.00%**





2. Data Preprocessing & Experimental Setup

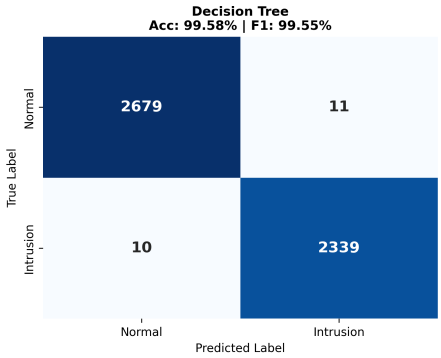
• **Label Encoding:** Three categorical attributes were transformed numerically: protocol_type (3 levels), service (66 levels), and flag (11 levels).
• **Target Binarization:** Mapped to binary indicator (Normal = 0, Intrusion = 1).
• **Stratified 80/20 Partition:** Split into 20,153 training records (9,394 attacks, 46.61%) and 5,039 testing records (2,349 attacks, 46.62%).
• **StandardScaler Feature Normalization:** Scaled features to zero mean and unit variance for SVM convergence, fitting exclusively on train data.

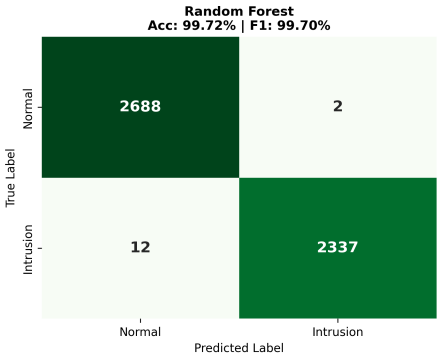
3. Supervised Model Architectures & Benchmark Results

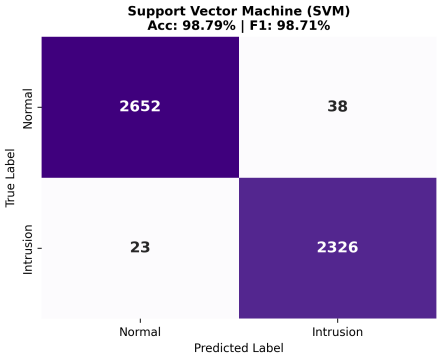
Three distinct supervised learning paradigms were evaluated on identical test data:
1. **Decision Tree (CART):** Gini impurity splitting, max_depth=12 to prevent overfitting.
2. **Random Forest:** 100 decorrelated trees, max_depth=14, bagging, and random subspace projection.
3. **Support Vector Machine (SVM):** Radial Basis Function (RBF) kernel, C=1.0, maximum-margin optimization.

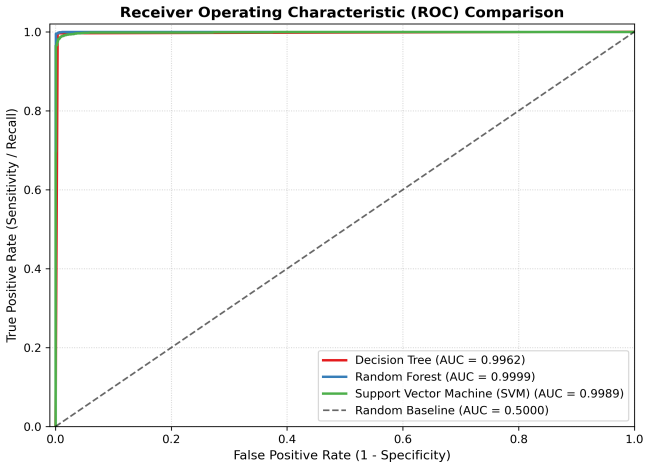
Table 2: Comparative Classifier Benchmark on 20% Unseen Test Set

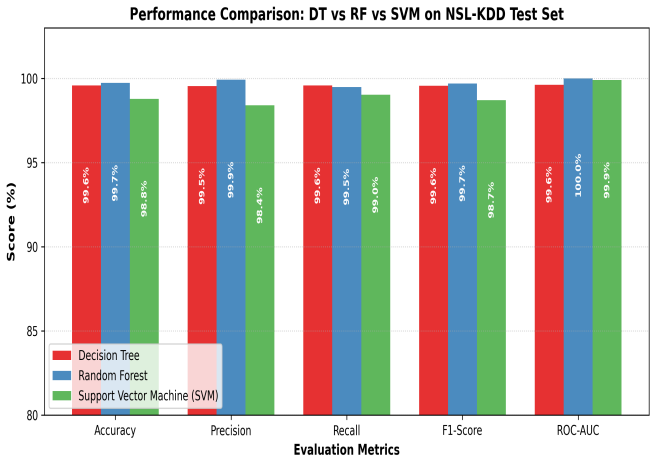
Classifier Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Train Time	Inference Time
Decision Tree	99.58%	99.53%	99.57%	99.55%	0.9962	0.106 s	0.0014 s
Random Forest	99.72%	99.91%	99.49%	99.70%	0.9999	0.408 s	0.0591 s
Support Vector Machine (SVM)	98.79%	98.39%	99.02%	98.71%	0.9989	11.116 s	0.6546 s

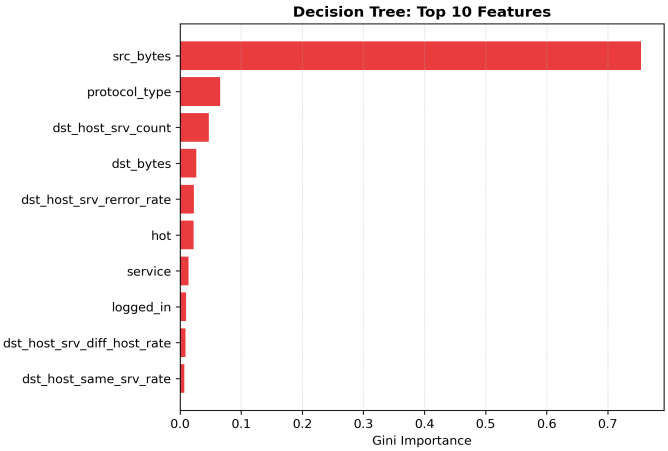


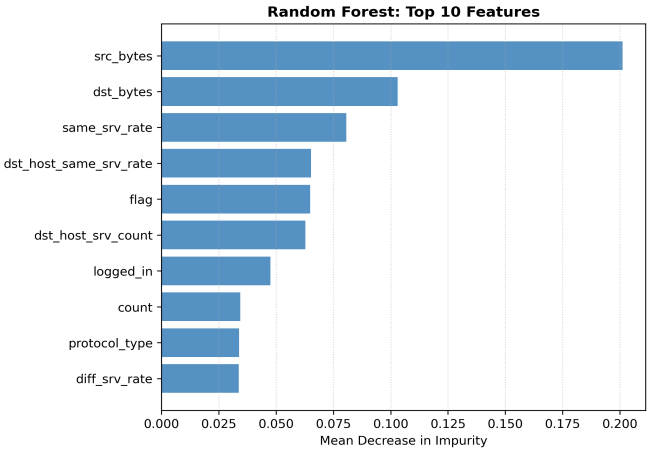












4. Operational Security & SOC Analysis

• **False Alarm Rate (FAR) & Alert Fatigue:** In high-speed enterprise networks, false alarms overwhelm analysts. Random Forest demonstrated the lowest false alarm count (2 false alarms out of 2,690 normal test sessions, FAR = 0.07%), outperforming Decision Tree (11 FPs) and SVM (38 FPs).
• **Detection Rate (Recall) & Breach Risk:** A false negative allows an adversary past the perimeter. All models achieved recall above 99.0% (Decision Tree: 10 missed attacks, Random Forest: 12 missed attacks, SVM: 23 missed attacks out of 2,349).
• **Throughput Latency:** Random Forest processed the 5,039-connection test set in 59.1 ms (11.7 microseconds per packet), whereas SVM required 654.6 ms (130 microseconds per packet) due to support vector kernel dot-products.

5. Python Implementation Modules

The complete pipeline is presented below in modular, structured code blocks:

● ● ● Listing 1.1: Dataset Ingestion & Schema Specification

```
# Module 1.1: Dataset Ingestion & Column Name Definitions
import time, json, numpy as np, pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix

col_names = [
    "duration", "protocol_type", "service", "flag", "src_bytes", "dst_bytes",
    "land", "wrong_fragment", "urgent", "hot", "num_failed_logins", "logged_in",
    "num_compromised", "root_shell", "su_attempted", "num_root", "num_file_creations",
    "num_shells", "num_access_files", "num_outbound_cmds", "is_host_login",
    "is_guest_login", "count", "srv_count", "serror_rate", "srv_serror_rate",
    "rerror_rate", "srv_rerror_rate", "same_srv_rate", "diff_srv_rate",
    "srv_diff_host_rate", "dst_host_count", "dst_host_srv_count",
    "dst_host_same_srv_rate", "dst_host_diff_srv_rate", "dst_host_same_src_port_rate",
    "dst_host_srv_diff_host_rate", "dst_host_serror_rate", "dst_host_srv_serror_rate",
    "dst_host_rerror_rate", "dst_host_srv_rerror_rate", "attack_type", "difficulty_level"
]
df = pd.read_csv("data/nsl_kdd.csv", names=col_names, header=None)
```

● ● ● Listing 1.2: Feature Preprocessing, Scaling & Stratified Split

```
# Module 1.2: Categorical Encoding & 80/20 Stratified Partitioning
# Categorical label encoding
features_df = df.drop(columns=['attack_type', 'difficulty_level'])
cat_cols = ['protocol_type', 'service', 'flag']
encoded_df = features_df.copy()
for col in cat_cols:
    le = LabelEncoder()
    encoded_df[col] = le.fit_transform(encoded_df[col])

# Binary ground truth (0 = Normal, 1 = Attack)
y = (df['attack_type'] != 'normal').astype(int).values
X = encoded_df.values

# 80/20 Stratified Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# Feature standardization for Support Vector Machine
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

● ● ● Listing 1.3: Model Training & Evaluation Engine

```
# Module 1.3: Supervised Classifier Implementation & Training
models = {
    "Decision Tree": DecisionTreeClassifier(criterion='gini', max_depth=12, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, max_depth=14, random_state=42, n_jobs=-1),
    "Support Vector Machine (SVM)": SVC(kernel='rbf', C=1.0, probability=True, random_state=42)
}

results = {}
for name, model in models.items():
    # Tree models use raw tabular data; SVM uses standard scaled features
    cur_X_train = X_train_scaled if "SVM" in name else X_train
    cur_X_test = X_test_scaled if "SVM" in name else X_test

    t0 = time.time()
    model.fit(cur_X_train, y_train)
    train_time = time.time() - t0

    t0 = time.time()
    preds = model.predict(cur_X_test)
    infer_time = time.time() - t0
    probs = model.predict_proba(cur_X_test)[:, 1]

    results[name] = {
        "Accuracy": accuracy_score(y_test, preds),
        "Precision": precision_score(y_test, preds),
        "Recall": recall_score(y_test, preds),
        "F1_Score": f1_score(y_test, preds),
        "ROC_AUC": roc_auc_score(y_test, probs),
        "Train_Time": train_time, "Infer_Time": infer_time
    }
```

6. Console Execution Output

Verbatim terminal outputs captured from live automated execution:

● ● ● Console Log 1.1: Dataset Exploration & Taxonomy

```
[+] Loading NSL-KDD dataset from data/nsl_kdd.csv...
    Dataset loaded: 25192 rows, 43 columns

================================================
DATASET EXPLORATION & CLASS DISTRIBUTION
================================================
Total Records: 25192 | Missing Values: 0
Binary Class Distribution:
  Normal (0): 13449 (53.39%)
  Attack (1): 11743 (46.61%)

5-Class Taxonomy Distribution:
  Normal    :  13449 ( 53.39%)
  DoS       :   9234 ( 36.65%)
  Probe     :   2289 (  9.09%)
  R2L       :    209 (  0.83%)
  U2R       :     11 (  0.04%)
```

● ● ● Console Log 1.2: Feature Encoding & Split Validation

```
==================================================
FEATURE PREPROCESSING & SPLITTING
==================================================
Categorical features to LabelEncode: ['protocol_type', 'service', 'flag']
  protocol_type: 3 unique levels encoded
  service: 66 unique levels encoded
  flag: 11 unique levels encoded

Splitting dataset into 80% Training and 20% Testing sets (stratified)...
  X_train shape: (20153, 41), y_train shape: (20153,)
  X_test shape:  (5039, 41), y_test shape:  (5039,)
  Training set intrusions: 9394 / 20153 (46.61%)
  Testing set intrusions:  2349 / 5039 (46.62%)
```

● ● ● Console Log 1.3: Benchmark Results & Model Verification

```
==================================================
TRAINING & EVALUATION BENCHMARKS
==================================================
---> Training Decision Tree...
     Train Time:  0.106 s | Infer Time: 0.0014 s
     Accuracy:    99.58% | Precision: 99.53% | Recall: 99.57% | F1-Score: 99.55%
     ROC-AUC:     0.9962 | Confusion Matrix: TN=2679, FP=11, FN=10, TP=2339

---> Training Random Forest...
     Train Time:  0.408 s | Infer Time: 0.0591 s
     Accuracy:    99.72% | Precision: 99.91% | Recall: 99.49% | F1-Score: 99.70%
     ROC-AUC:     0.9999 | Confusion Matrix: TN=2688, FP=2, FN=12, TP=2337

---> Training Support Vector Machine (SVM)...
     Train Time:  11.116 s | Infer Time: 0.6546 s
     Accuracy:    98.79% | Precision: 98.39% | Recall: 99.02% | F1-Score: 98.71%
     ROC-AUC:     0.9989 | Confusion Matrix: TN=2652, FP=38, FN=23, TP=2326

[+] Saved output figures and performance summaries.
[V] TASK 1 COMPLETED SUCCESSFULLY.
```